

Comparing sequences: exact, heuristic, learned

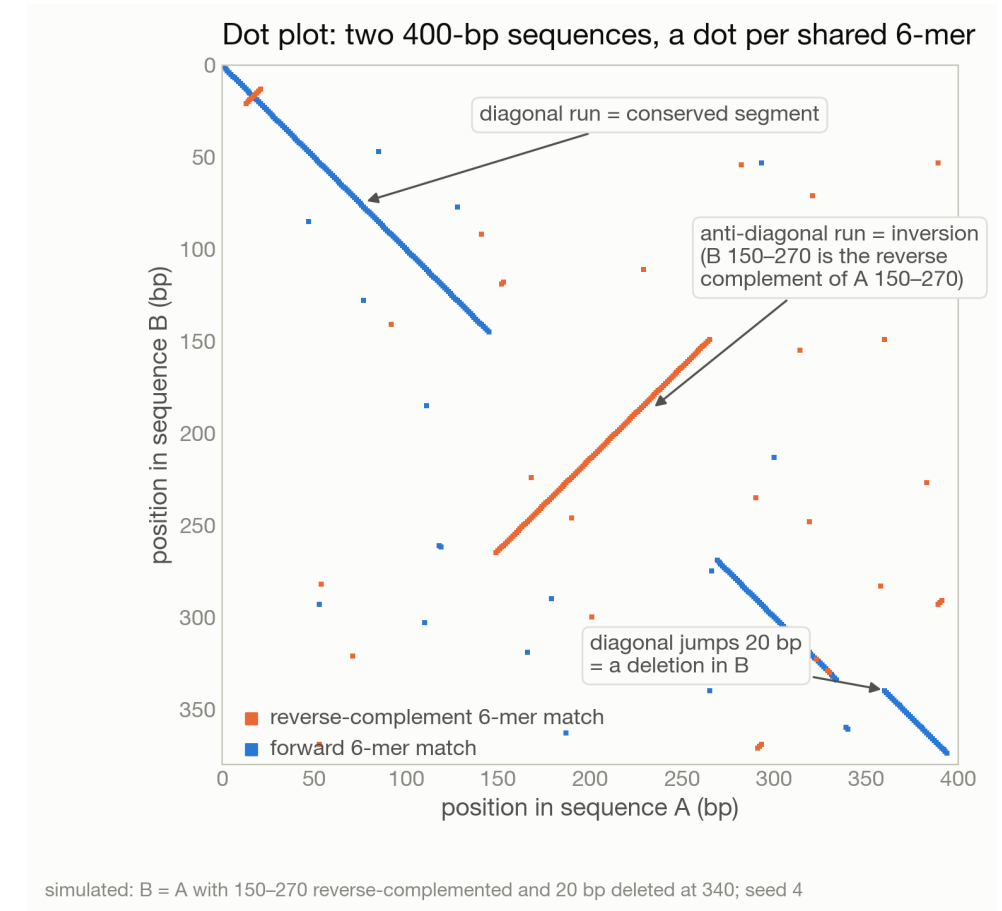
- BIB — Bioinformatics for Bioengineering / Biyomühendislik için Biyoinformatik
- Week 4 · Block A "The floor"
- Dizi karşılaştırma: kesin, sezgisel, öğrenilmiş
- Yasin Kaymaz, PhD · Ege University, Dept. of Bioengineering

Ad of the Week #4

- Skills Ledger tick this week: "sequence comparison and its guarantees"

The dot plot: one picture, then we move on

- Two sequences on two axes, a dot wherever the letters match
- A diagonal run = conserved segment (korunmuş bölge)
- A gap in the diagonal = an indel
- An anti-diagonal run = an inversion (ters çevrilme / inversiyon)
- That is the whole idea. The algorithm and the pseudocode are gone.
- You will meet this picture again as a genome-to-genome plot in any assembly paper



Dot plot of two simulated 400-bp sequences: diagonal = conserved, anti-diagonal = inversion (B 150-270 reverse-complemented), diagonal jump = 20-bp deletion

The only algorithm with a proof attached

- For the next 26 minutes nobody types anything. Paper and pen only.
- We fill ONE local alignment matrix (yerel hizalama matrisi), together, cell by cell
- The sequences are the ones from last year's deck, deliberately: $X = \text{GATAGA}$, $Y = \text{GATTACA}$
- The scoring scheme is FIXED and non-negotiable: match +2, mismatch -1, gap -2
- Local means: the matrix has a floor at zero. A cell can never go negative.
- Everyone writes. I will call on rows.

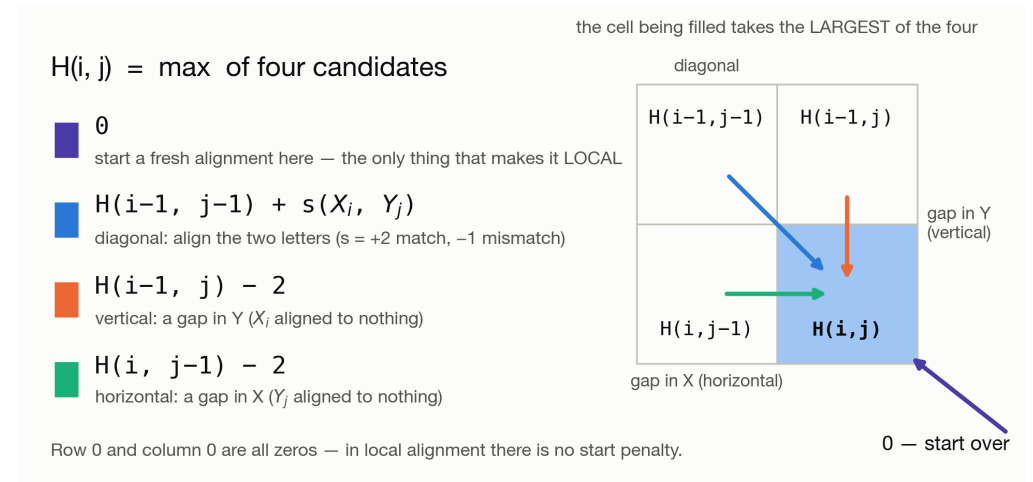
	-	G	A	T	T	A	C	A
-	0	0	0	0	0	0	0	0
G	0							
A	0							
T	0							
A	0							
G	0							
A	0							

$X = \text{GATAGA}$ (rows) · $Y = \text{GATTACA}$ (columns) · match +2 · mismatch -1 · gap -2 · floor at 0

$X = \text{GATAGA}$ (rows) vs $Y = \text{GATTACA}$ (columns); row 0 and column 0 pre-filled with zeros

The recurrence, written once

- $H(i,j)$ = the best score of any alignment ENDING at X_i paired with Y_j
- Four candidates, take the largest:
 - 0 — start a fresh alignment here (this is the only thing that makes it local)
 - $H(i-1, j-1) + s(X_i, Y_j)$ — a diagonal move: align the two letters
 - $H(i-1, j) - 2$ — a vertical move: a gap in Y
 - $H(i, j-1) - 2$ — a horizontal move: a gap in X
- $s = +2$ if the letters match, -1 if they do not
- Row 0 and column 0 are all zeros. Always. In local alignment there is no start penalty.



$H(i,j) = \max$ of four candidates: 0, diagonal + s , vertical -2 , horizontal -2

Fill it: rows 1 and 2, together

- Row 1 ($X_1 = G$): 2 0 0 0 0 0 0
- Row 2 ($X_2 = A$): 0 4 2 0 2 0 2
- Watch what the zero floor does:
most of row 1 is zero, not negative
- $H(2,2) = 4$ because $H(1,1) = 2$ and A/A is a match: $2 + 2$
- $H(2,3) = 2$ came HORIZONTALLY from $H(2,2)$: $4 - 2 = 2$, not from the diagonal
- Call out the value, then the DIRECTION it came from. Both matter.

	—	G	A	T	T	A	C	A
—	0	0	0	0	0	0	0	0
G	0	2	0	0	0	0	0	0
A	0	0	4	2	0	2	0	2
T	0							
A	0							
G	0							
A	0							

Row 2 ($X_2 = A$): 0 4 2 0 2 0 2 — $H(2,3) = 2$ came HORIZONTALLY from $H(2,2) = 4$

Rows 1 and 2 filled; each non-zero cell carries an arrow to the neighbour it came from

The completed matrix

- Row 3 (T): 0 2 6 4 2 1 0
- Row 4 (A): 0 2 4 5 6 4 3
- Row 5 (G): 2 0 2 3 4 5 3
- Row 6 (A): 0 4 2 1 5 3 7
- The maximum anywhere in the matrix is 7, at H(6,7)
- In a LOCAL alignment you start the traceback at the maximum — not at the corner
- Here they happen to be the same cell. That is a coincidence, and I want you to notice it is a coincidence.

	–	G	A	T	T	A	C	A
–	0	0	0	0	0	0	0	0
G	0	2	0	0	0	0	0	0
A	0	0	4	2	0	2	0	2
T	0	0	2	6	4	2	1	0
A	0	0	2	4	5	6	4	3
G	0	2	0	2	3	4	5	3
A	0	0	4	2	1	5	3	7

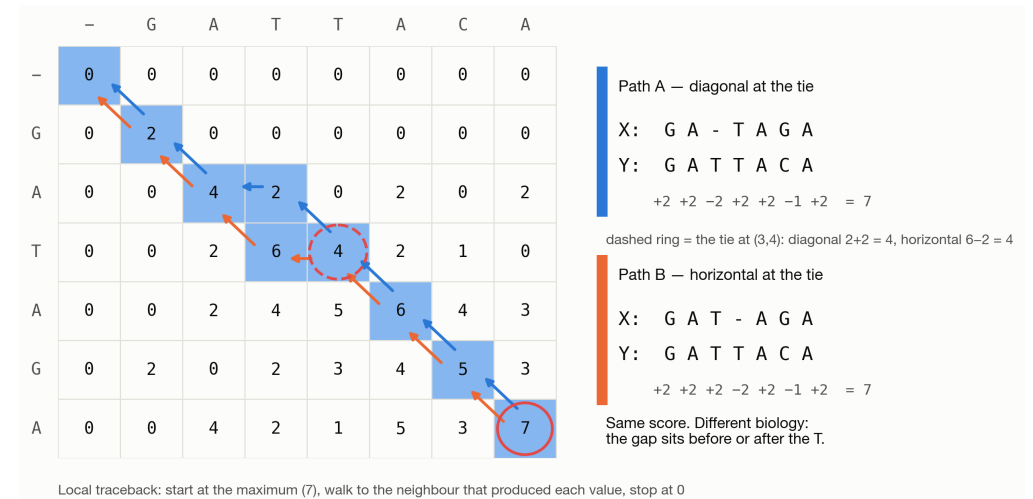
$X = \text{GATAGA (rows)} \cdot Y = \text{GATTACA (columns)} \cdot \text{match} +2 \cdot \text{mismatch} -1 \cdot \text{gap} -2 \cdot \text{floor at } 0$

$\text{max} = 7, \text{ at } H(6,7) \text{ — the local traceback starts here}$

The completed local-alignment matrix; maximum 7 at H(6,7), ringed

Traceback — and the tie that matters

- From (6,7)=7 walk backwards, at each step to the neighbour that produced the value
- One optimal answer:
 - X: G A - T A G A
 - Y: G A T T A C A
 - Score check: $+2 +2 -2 +2 +2 -1 +2 = 7$ ✓
- But at cell (3,4)=4 there is a tie: diagonal $2+2 = 4$, and horizontal $6-2 = 4$
- Take the other branch and you get an EQUALLY optimal answer, score also 7:
 - X: G A T - A G A
 - Y: G A T T A C A
- Same score. Different biology. Remember this in eight slides' time.



What you just bought, and where it actually lives

- The guarantee: nothing scores higher than 7 under this scoring scheme. Proven, not tested.
- Cost: $O(mn)$ time and memory. For two 3-kb sequences that is 9 million cells. For a genome it is impossible.
- So nobody runs it on a genome — they run it on the small pieces a heuristic hands them:
 - minimap2 and BWA-MEM: seed with minimizers, then Smith–Waterman the gaps between seeds
 - The CIGAR string in every SAM/BAM file you will ever open is the output of a traceback
 - Dorado's basecaller uses DP to align its own output to a reference for consensus
- Exact algorithms did not lose. They moved inside the heuristics.

Affine gaps: why one 5-base gap beats five 1-base gaps

- Linear gap penalty: five separate 1-base gaps cost the same as one 5-base gap ($5 \times -2 = -10$)
- Biologically that is wrong. One polymerase slip removes five bases in one event.
- Affine gap penalty (afin boşluk cezası): cost = $\text{gap_open} + (\text{length} \times \text{gap_extend})$
 - Typical: open -11 , extend $-1 \rightarrow$ a 5-base gap costs -16 , five 1-base gaps cost -60
- Every real aligner uses affine. BLAST default for BLASTP is open 11, extend 1.
- Two extra matrices are needed to compute it (Gotoh 1982). You will not do that by hand.

one 5-base gap (one event)	
X: A T G - - - C T A	linear, -2 per gap base: $5 \times (-2) = -10$
Y: A T G T T G C A C T A	affine, open -11 + extend -1 per base: $-11 - 5 \times 1 = -16$
five 1-base gaps (five events)	
X: A - T - G - C - T - A	linear, -2 per gap base: $5 \times (-2) = -10$
Y: A T T T G G C C T T A	affine, open -11 + extend -1 per base: $5 \times (-11 - 1) = -60$
Y differs between the two — only the gap columns (red) are being priced; every letter column is a match in both, so the letters are greyed.	
Same six letters of X, same five missing bases. Linear cannot tell the two apart; affine (BLASTP default: open 11, extend 1) prefers the single event by 44 points.	

Linear penalty cannot tell one 5-base gap from five 1-base gaps (-10 both); affine (open -11 , extend -1) scores them -16 vs -60

Indel representation is ambiguous

- You saw it yourself twenty minutes ago: the tie at cell (3,4)
- Two placements of the same gap, identical score, different coordinate
- In a repeat, it is worse: AAAA → AAA — WHICH A was deleted? Four legal answers.
- Consequence: the SAME variant can be written several ways in a VCF
- This is why bcftools norm exists — left-align and normalise indels before you compare anything
- This is why comparing two VCF files with diff is meaningless, and why benchmarking uses hap.py / vcfeval
- Banked here. Cashed in Week 13, when you write a sentence somebody acts on.

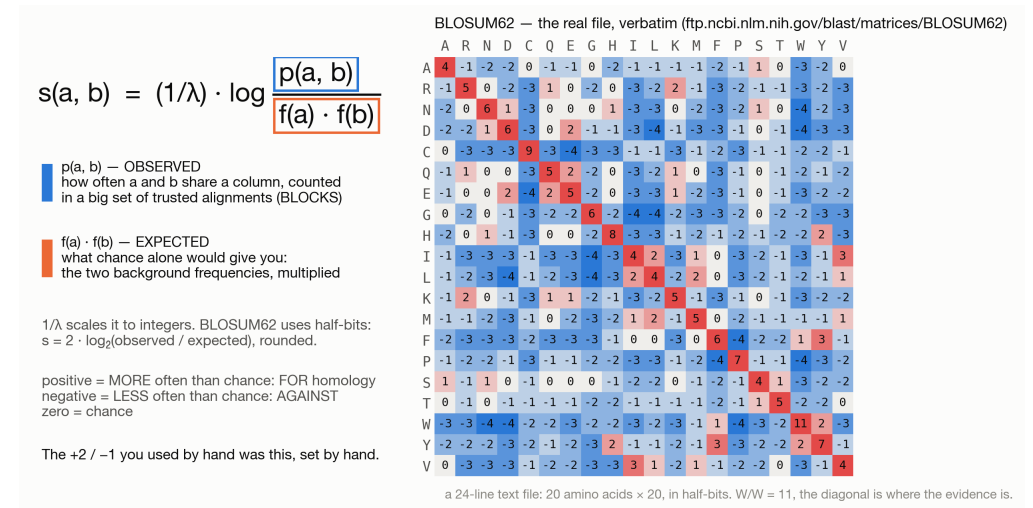
reference chr1:99–107	#CHROM	POS	REF	ALT	the deleted bases
G T A A A A C G 99 100 101 102 103 104 105 106 107	chr1	100	TAAA	T	form 1: delete the first three A's
G T A A A A C G 99 100 101 102 103 104 105 106 107	chr1	101	AAAA	A	form 2: delete the middle three
G T A A A A C G 99 100 101 102 103 104 105 106 107	chr1	102	AAAA	A	form 3: delete the last three
G T A A A A C G 99 100 101 102 103 104 105 106 107	chr1	100	TAAAAAC	TAAC	form 4: untrimmed — shared end bases kept
G T A A A A C G 99 100 101 102 103 104 105 106 107	chr1	100	TAAA	T	bcftools norm: left-aligned, parsimonious — ONE line

All five lines describe the same haplotype G T A A C G. 'diff' sees four different variants; 'bcftools norm' sees one. Rule: samtools.github.io/bcftools/bcftools.html#norm

*One 3-base deletion in an A5 homopolymer, four legal VCF spellings,
one bcftools norm form (chr1 100 TAAA T)*

Substitution scores: $\log(\text{observed} / \text{expected})$

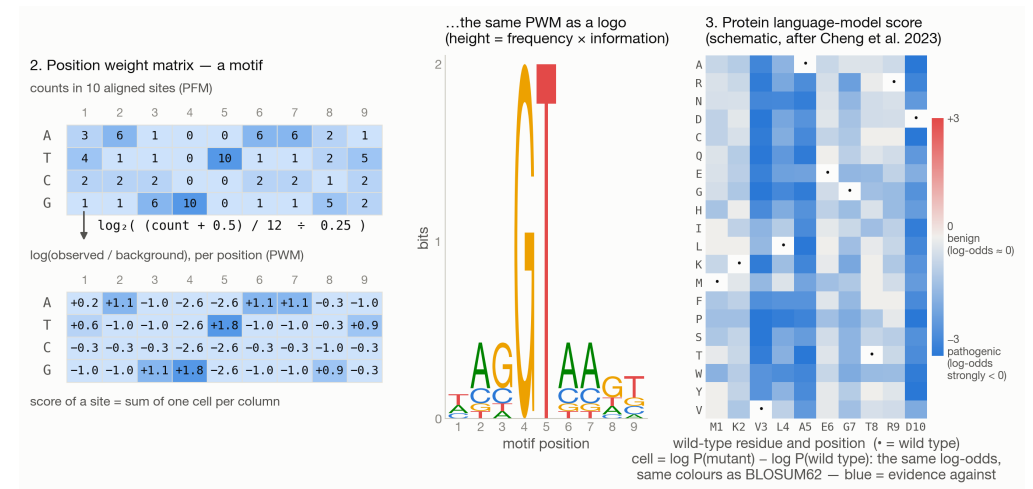
- Question: how much evidence is it that I see a W aligned to a W?
- Answer: compare what you OBSERVE in real alignments to what you would EXPECT by chance
 - $s(a,b) = (1/\lambda) \cdot \log(p_{\text{observed}}(a,b) / (f_a \cdot f_b))$
- Positive score = seen more often than chance = evidence FOR homology
- Negative score = seen less often than chance = evidence AGAINST
- This object has a name in statistics: a log-likelihood ratio (log-olabilirlik oranı)
- The +2 and –1 you used by hand were a crude, hand-set version of this



$s(a,b) = (1/\lambda) \cdot \log(\text{observed}/\text{expected})$, and the real BLOSUM62 file (NCBI, verbatim) as a 20×20 heatmap in half-bits

The same object, three times — this is a deposit

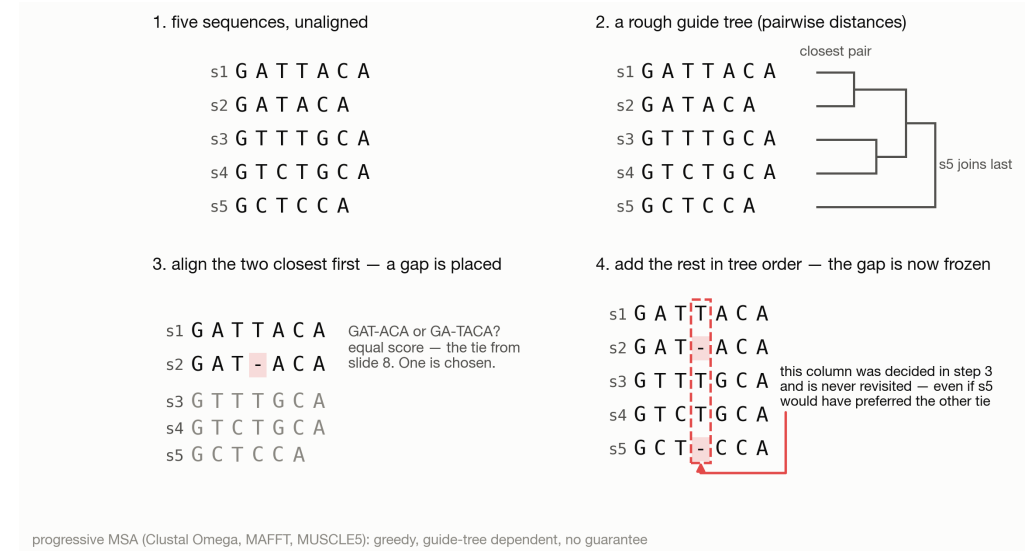
- 1. Substitution matrix (1992). BLOSUM62 entry = $\log(\text{observed pair frequency} / \text{expected})$
- 2. Position weight matrix (konum ağırlık matrisi): per position, $\log(\text{observed} / \text{background})$
 - A PWM is a linear model over one-hot DNA — a conv layer's first filters ARE PWMs
- 3. Protein language model. ESM-1v / AlphaMissense: $\log P(\text{mutant}) - \log P(\text{wild-type})$, per position
 - From a model trained on 250 million sequences
- Different decades, different data, different compute. Identical mathematical form.
- You meet #2 again in Week 11 and #3 again in Week 13. Write that down.



The same log-odds object three times: PFM→PWM, the PWM as a sequence logo, and a per-position missense-effect heatmap (schematic, after Cheng et al. 2023)

Multiple sequence alignment, defined and then parked

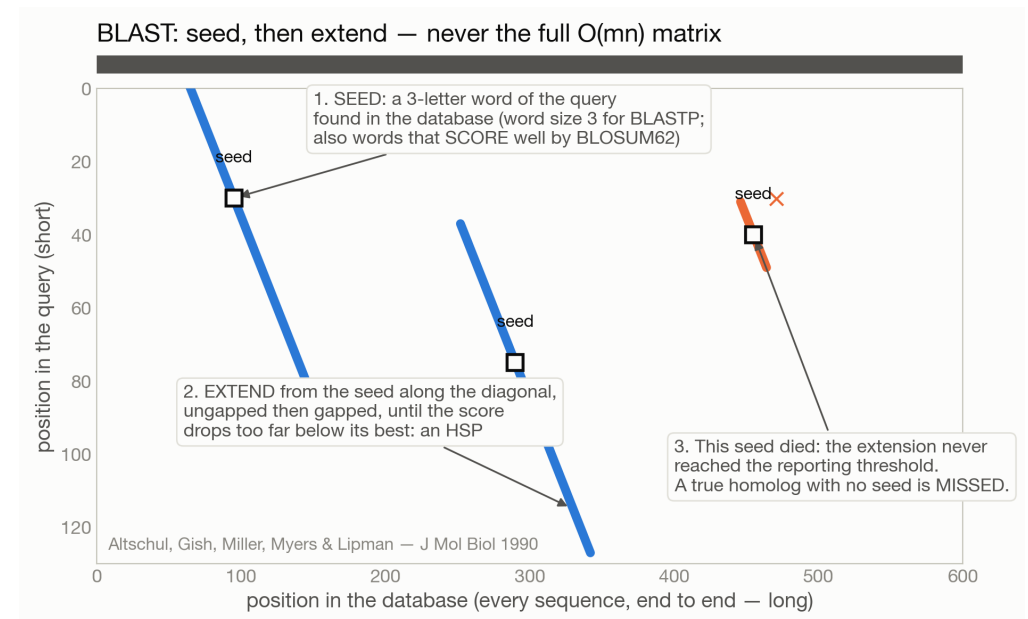
- MSA (çoklu dizi hizalaması): align N sequences simultaneously, one column per homologous position
- Doing it exactly is $O(L^N)$. Nobody does it exactly.
- Progressive alignment: build a rough tree, align the two closest, then add the next, and so on
 - It is greedy (açgözlü): an error made early is never revisited
 - The result therefore depends on the guide tree (kılavuz ağaç), which depends on the rough distances
- Named, not run: Clustal Omega, MAFFT, MUSCLE5, and for very large sets, mmseqs-based pipelines
- Where you meet one: as input to a structure predictor
 - MSA depth drives AlphaFold's confidence (Week 12)



Progressive alignment: five sequences, guide tree, closest pair first, rest added in tree order; the gap placed in step 3 is frozen

BLAST: seed, then extend

- The problem: Smith–Waterman against a whole database is impossible. So do not do it.
- Seed: chop the query into short words. Default word size 3 for BLASTP, 11 or 28 for BLASTN.
- Neighbourhood words: seed on words that SCORE well against the query word, not just identical ones
 - For proteins — this is where BLOSUM62 is actually used
- Extend: only where a seed hits, extend ungapped, then gapped, then compute a real local alignment
- Result: ~50× faster than Smith–Waterman, and it can MISS things. There is no proof.
- Published, characterised failure behaviour — which is the honourable kind of no-guarantee



BLAST: three word seeds, two extend into high-scoring segment pairs, one dies (Altschul et al., J Mol Biol 1990)

The E-value is not a p-value, and it moves

- E-value (E-değeri): the number of hits this good you would expect BY CHANCE in a database of this size
- $E = K \cdot m \cdot n \cdot e^{(-\lambda S)}$ — the only thing you need from this: E scales with database size
- So: the SAME alignment, searched against a bigger database next year, gets a WORSE E-value
- Nothing about the biology changed. The denominator did.
- Practical rule: an E-value without a database name and a date is not a result
- This is a version stamp lesson wearing a statistics costume — GENOMES row S

The same HSP — identical letters, identical score — and its E-value moves seven orders of magnitude

database searched	size (residues, n)	identity	score	E-value
nr, 2010	$\sim 3.5 \times 10^9$	42 % over 130 aa	62 bits	1.5×10^{-7}
nr, 2026	$\sim 3 \times 10^{11}$	42 % over 130 aa	62 bits	1.3×10^{-5}
custom, 200 proteins	$\sim 7 \times 10^4$	42 % over 130 aa	62 bits	3.0×10^{-12}

illustrative, not a real run: $E = m \cdot n \cdot 2^{(-S')}$ with $m = 200$ aa, $S' = 62$ bits; database sizes are approximate. Nothing about the biology changed — n did.

The same HSP (42 % identity, 62 bits) against three databases: E-value moves from 3×10^{-12} to 1×10^{-5} with database size alone

Identity is not coverage

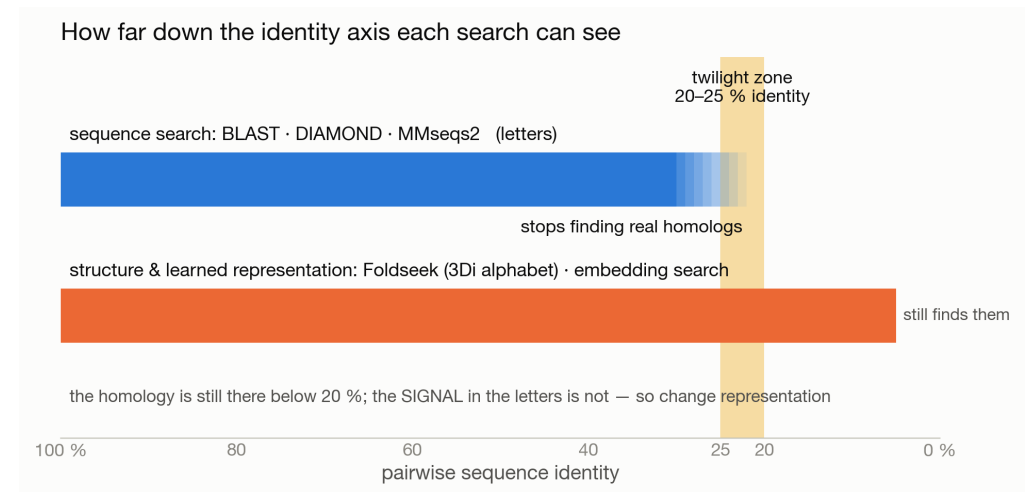
- Real BLASTP tabular output, -outfmt "6 qseqid sseqid pident length qcovs evaluate bitscore"
- Query = viral cyclin. Subject set = human cyclins and everything else that hit.
- Read the columns out loud. Two of them are doing different jobs:
 - pident — percent identity (özdeşlik yüzdesi): of the columns that were ALIGNED, how many matched
 - qcovs — query coverage (sorgu kapsamı): what fraction of your query took part at all
- The classic misreading: 98% identity over 11% of the query
- That is a shared domain, or a low-complexity stretch. It is not "these are the same protein".

Scale: BLAST is no longer the fast option

- BLAST is slow, and faster tools exist. That part is an engineering problem, and it is solved.
- DIAMOND — up to $\sim 10^4\times$ faster than BLASTX for protein search (Buchfink et al., Nat Methods 2021)
- MMseqs2, and MMseqs2-GPU (Nat Methods 2025): $\sim 16\times$ faster than BLAST on a single L40S
- Production metagenome and proteome annotation moved to these years ago
- Named, not installed. You will not run these this semester.

The ceiling: the twilight zone

- BLAST's ceiling is biological, not computational — an information problem, and it is not solved
- Below roughly 20–25% identity — the "twilight zone" — sequence search stops finding real homologs
- The homology is still there. The SIGNAL is not, in that representation.
- So change representation: Foldseek searches a learned structural alphabet (3Di)
- Embedding search does the same trick with protein-language-model vectors
- Same algorithm, different representation.
- Named, not installed.



Sequence search fades out in the 20-25 % twilight zone; Foldseek 3Di and embedding search keep going below it

The table. Photograph this one.

- Exact (kesin) · Heuristic (sezgisel) · Learned (öğrenilmiş)
- Examples: Smith–Waterman · BLAST, DIAMOND, MMseqs2, minimap2 · ESM, AlphaMissense, Dorado, Foldseek's 3Di
- Every tool named for the rest of this semester gets sorted into this table.

	Guarantee	Cost	The question to ask
Exact (kesin) Smith–Waterman	Provably optimal — but only under the scoring scheme you chose	Expensive: $O(mn)$	“What scoring scheme, and who chose it?”
Heuristic (sezgisel) BLAST · DIAMOND · MMseqs2 · minimap2	None. But a characterised, published failure behaviour	Fast	“What does it systematically miss?”
Learned (öğrenilmiş) ESM · AlphaMissense · Dorado · Foldseek's 3Di	None. Failure behaviour depends on training data	Fast at inference; expensive to train	“What was it trained on, and is my case in it?”

Every tool named for the rest of the semester gets sorted into this table. Real tools are compounds — sort the STEPS, not the product.

Exact / Heuristic / Learned: guarantee, cost, and the question to ask

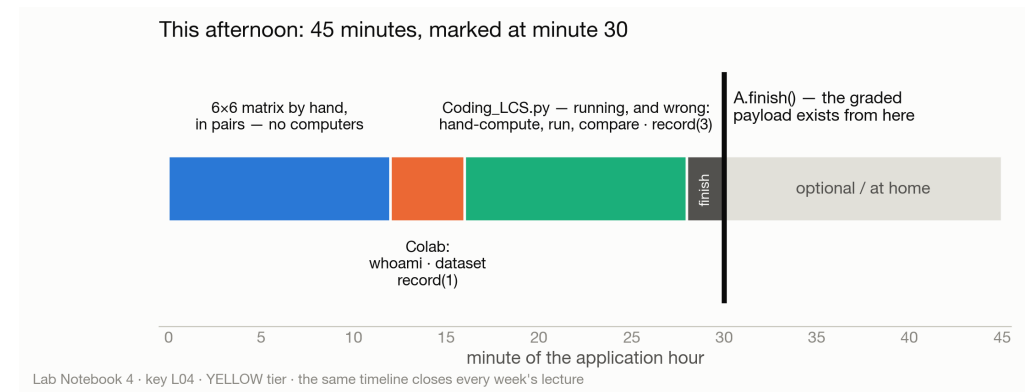
	Guarantee	Cost	Ask this
Exact	Provably optimal	$O(mn)$	Whose scheme?
Heuristic	None, published	Fast	Misses what?
Learned	None, data-bound	Fast to run	Trained on?

Sort these, out loud, right now

- Thirty seconds each, called from the room:
 - Your hand-filled matrix from this morning
 - blastp against nr
 - minimap2 mapping nanopore reads to GRCh38
 - Foldseek
 - Dorado, the basecaller that made every FASTQ you will open
 - MAFFT
- For each: which row, and what guarantee does it carry?
- Two of these are trick questions.

This afternoon, and what is being marked

- Application hour, 45 minutes. Hand, then machine, then audit — one alignment two ways.
- Minutes 0–12: a 6×6 matrix from YOUR seed, on paper, in pairs. No computers.
- Minutes 12–16: Colab. Your HAND answer goes in FIRST: `A.record(1, hand_score, hand_aligned_len)`
- Minutes 16–28: `Coding_LCS.py` — complete, running, and wrong. Hand-compute a 5-base pair, run, compare.
- Minute 30: `A.finish()`. Everything after minute 30 is optional and can be done at home.
- Lab Notebook 4, key L04, YELLOW tier. There is no BLASTP question. There is no record key 4.



Application hour: hand matrix 0-12, Colab 12-16, LCS audit 16-28, A.finish() at minute 30, the rest optional

Before you go

- Do: tick "sequence comparison and its guarantees" on your Skills Ledger
- Do: photograph the trichotomy table
- Read (30 min, no assessment): Altschul et al. 1990, first two pages only — doi:10.1016/S0022-2836(05)80360-2
- Banked today, cashed later: log-odds → Weeks 11 and 13 · indel ambiguity → Week 13 · MSA depth → Week 12
- Everything for this week is on EgeDers. GitHub is a mirror, not the source.
- Next week: from a molecule to a read. Bring your Design Notebook 1 nerves — it is issued in the application hour.